

Analysis pipeline for machine-learning transition state analysis

Programme write-up

PHAS0077 — Scientific Computing Individual Research Project

Yucheng Zhao

University College London, Department of Physics and Astronomy

MSc Scientific and Data Intensive Computing

<https://github.com/171172701a/mltsa-trypsin-msc2025-2026>

1 Problem statement and requirements

1.1 What the program is for

Drug–target residence time is increasingly recognised as a better predictor of *in vivo* efficacy than equilibrium affinity, but rationalising it requires knowing which molecular interactions decide whether a ligand escapes. Machine-learning transition state analysis addresses this by launching many unbiased molecular dynamics trajectories from the transition-state region, letting each commit to either the bound or the unbound basin, and then training a classifier to predict that committed outcome from a large set of collective variables. The coordinates the classifier relies on are the candidate mechanistic determinants; the whole approach therefore reduces to a supervised learning problem whose conclusions rest entirely on one component, the feature-importance measure.

The program described here implements that analysis for an ensemble of 173 benzamidine–trypsin dissociation trajectories, 99 of which rebound and 74 of which release. Its scientific output is reported in the accompanying dissertation. The concern of this document is the software: what it had to do, how it was built, whether it does it correctly, what it cost, and how far its results can be regenerated.

Two properties of the problem shaped every design decision, and both are silent failure modes: they inflate results without producing any visible symptom. Frames within a trajectory are strongly autocorrelated, and the analysis window is itself a free parameter.

1.2 Requirements

The input is a single HDF5 file containing a three-dimensional array of shape (173, 2500, 274): 173 independent trajectories, each of 2,500 frames, each frame described by 274 collective

variables representing minimum distances from the ligand to 223 protein residues, one calcium ion, and the 50 water molecules nearest the ligand at that frame. Each trajectory carries a binary outcome label and a continuous end-state score. The array holds 118,505,000 values, occupying approximately 0.95 GB in memory at double precision.

From this, five requirements followed.

R1 — Correctness under correlated sampling. Frames within a trajectory are strongly autocorrelated. Any procedure treating them as independent samples would produce optimistic and meaningless cross-validated scores, because a held-out frame can be recognised from its near-duplicate in the training set. The software had to make this class of error structurally impossible rather than merely avoided by convention.

R2 — Windowed aggregation. The analysis required comparing models fitted to different frame ranges. The software had to compute per-trajectory summaries over an arbitrary window without re-reading the full array each time.

R3 — Nested model selection at scale. Both the analysis window and the model hyperparameters had to be selected without reference to test data. With roughly 37 candidate windows, 2 hyperparameter configurations, 5 outer folds and 3 inner folds, a single run involves on the order of 10^3 model fits; with a 99-repeat permutation null layered on top, on the order of 10^5 .

R4 — Multiple interpretability measures. Three feature-importance measures (impurity, permutation, Shapley) had to be computed on identical fitted models and held-out data, so that disagreement between them could be attributed to the measures rather than to differences in fitting.

R5 — Exact reproducibility. Given a 12-hour runtime, it had to be possible to regenerate every reported figure and number without repeating the computation. In the event this requirement was exercised directly: every importance statistic in the accompanying dissertation was later recomputed from the stored arrays (Section 5.4).

2 Architecture and data flow

The pipeline is organised as five stages, each producing an artefact consumed by the next.

Stage 1 — Ingestion. The HDF5 file is opened with `h5py` and the label vector, group identifiers and end-state scores are read. Guard clauses reject a malformed array before anything downstream runs (Section 6.2). The run record additionally stores `mixed_label_groups`, the set of trajectory identifiers carrying more than one outcome label, and whether continuous end-state scores were present; the first is empty for the dataset used here. These two are recorded rather than enforced, so a reader of the outputs can check them after the fact.

Stage 2 — Windowed aggregation. For a candidate window w , the array is reduced along the frame axis to yield a matrix of shape (173, 274) — one mean feature vector per trajectory. This is the pivotal design decision in the whole pipeline and it serves two purposes simultaneously.

Scientifically, it is the correctness requirement R1: once a trajectory is a single row, there is no mechanism by which frames of one trajectory can appear on both sides of a split, because a trajectory cannot be divided.

Computationally, it reduces the working matrix from $173 \times 2,500 \times 274 \approx 1.2 \times 10^8$ values to

$173 \times 274 \approx 4.7 \times 10^4$, a reduction of three orders of magnitude. Every downstream operation — cross-validation, model fitting, permutation importance, bootstrap resampling — operates on the small matrix. A frame stride of 10 is applied during aggregation, so that windows are summarised from every tenth frame; this reduces the arithmetic without materially changing a mean over a window of 10 to 200 frames.

The consequence is that the expensive part of the pipeline is not the data volume but the number of model fits, which is the opposite of what a hundred-million-element input array suggests. The design follows from this: there is no out-of-core processing anywhere in the pipeline, and the engineering effort visible in the code is directed at controlling the combinatorics of the search rather than the volume of the data.

Stage 3 — Grouped splitting. All partitions are produced by `StratifiedGroupKFold` with the trajectory identifier as the grouping variable, preserving the 99:74 class ratio across folds while guaranteeing group integrity.

Stage 4 — Nested selection and fitting. For each outer fold, the inner loop scores candidate configurations using only inner splits of the outer training partition — in two stages rather than exhaustively, as Section 4.1 describes — selects the best by mean balanced accuracy, refits on the full outer training partition, and records predictions and importances on the outer test partition alone.

Stage 5 — Serialisation. The nested search writes its results as JSON (`run_metadata.json`, `summary.json`) together with PNG figures, and persists the final fitted model with `joblib`. The two-window analysis writes its own JSON record (`reproducibility.json`), the per-seed importance matrices as a NumPy archive (`fi_per_seed.npz`) and the per-feature rankings as CSV.

The five stages and their principal artefacts are shown in Figure 1.

3 Correctness and leakage control

This section describes the largest single risk to the validity of the analysis and how the software addresses it.

3.1 Why the naïve approach fails

The obvious implementation treats each of the 432,500 frames as an independent sample and applies a standard train/test split. This is wrong in a way that does not announce itself: the resulting accuracy is high, the confusion matrix looks healthy, and nothing in the output indicates a problem. Consecutive frames of an MD trajectory are separated by a short save interval and are correspondingly close in feature space, so a model that has seen frame n can classify frame $n + 1$ without having learned anything transferable. The error inflates the reported score by an unknown but potentially large margin, and it is invisible to inspection.

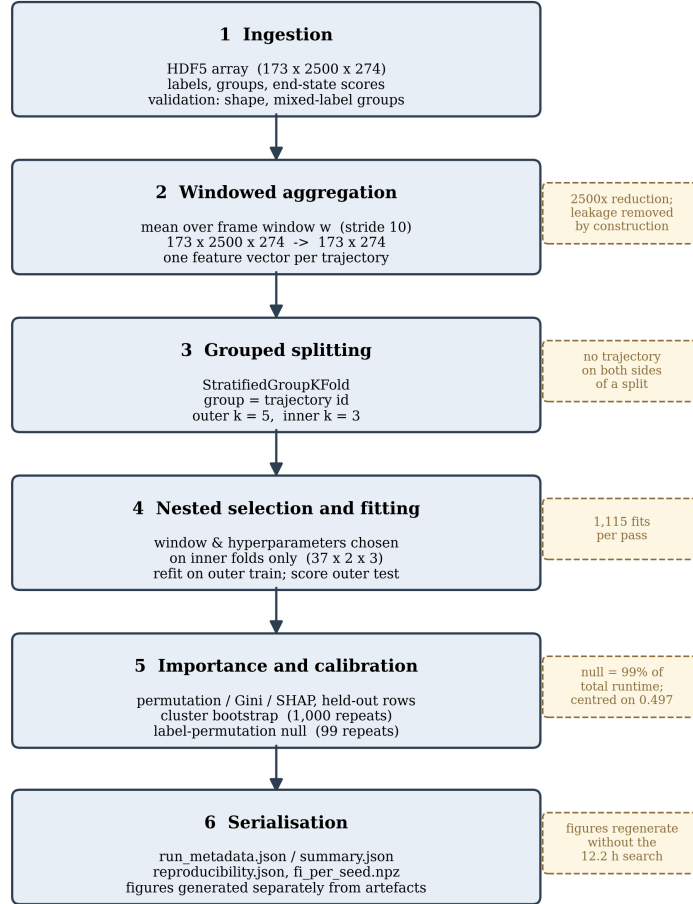


Figure 1: Data flow through the analysis pipeline. Annotations on the right give the consequence of each design decision: the aggregation stage reduces the working array by a factor of 2,500 and simultaneously removes the frame-level leakage risk by construction, while the null baseline in stage 5 accounts for approximately 99% of total runtime and returns a distribution centred on 0.497, confirming that the grouping logic behaves as intended.

3.2 Three layers of defence

Layer 1 — Structural. Aggregating each trajectory to a single row (Stage 2) eliminates the failure mode by construction. A structural guarantee is preferred to careful splitting because it cannot be undone by a later edit.

Layer 2 — Grouped cross-validation. All splits use `StratifiedGroupKFold` grouped by trajectory. This is redundant with Layer 1 for the aggregated matrix, but it holds if the aggregation is ever relaxed, and it makes the intent explicit in the code.

Layer 3 — Nested selection. A subtler leak arises when a configuration is chosen by inspecting test performance. With 37 candidate windows and 5 folds, selecting the best window on test data would guarantee an optimistic result even with perfect group separation. Window and hyperparameter selection therefore occur entirely within inner splits of each outer training partition (Algorithm 1 of the dissertation). No test trajectory influences either choice.

3.3 Empirical verification of the implementation

The three layers above are design arguments. They do not, by themselves, demonstrate that the implementation is correct. The pipeline therefore includes a test that would fail if leakage were present anywhere in the chain.

The entire nested procedure is rerun 99 times with the trajectory labels randomly permuted, each repeat searching the complete candidate grid. If the implementation is sound, a model trained on shuffled labels cannot exceed chance performance, and the distribution of scores should be centred on 0.5.

The observed null distribution has mean 0.497 and standard deviation 0.040 — centred on chance to within a tenth of a standard deviation. This is the single strongest evidence available that the splitting, aggregation and nesting logic are implemented correctly. A pipeline leaking information between folds produces a null distribution displaced above 0.5, because the leaked signal survives label permutation while the genuine signal does not.

Two design choices make this a stringent test rather than a formality. First, the null repeats search the *full* candidate grid (`null_use_full_grid = true`), so the null model enjoys exactly the same freedom to overfit through selection as the real one; restricting the null’s search would have made the comparison flattering and the test weak. Second, the permutation is applied at the trajectory level, matching the grouping structure of the real analysis.

The null baseline therefore functions as an integration test of the pipeline as well as a statistical control, and it is useful in that role independently of whether a p -value is wanted.

3.4 Confidence intervals respecting group structure

For the same reason that splits must be grouped, resampling must be. Confidence intervals are computed by cluster bootstrap: whole trajectories are resampled with replacement, 1,000 times. A naïve bootstrap over rows would understate uncertainty by treating correlated observations as independent.

4 Performance and execution

4.1 The size of the computation

The candidate grid contains 37 windows (three window lengths at twelve positions each, plus the full trajectory) and two hyperparameter configurations. An exhaustive inner search would evaluate every pairing, but the implementation screens in two stages instead. Stage 1 evaluates all 37 candidates using only the most conservative model in the grid, at $37 \times 3 = 111$ fits per outer fold. Stage 2 then tries the remaining configuration on only the eight highest-scoring candidates, at $8 \times 1 \times 3 = 24$ further fits. With one refit on the full outer training partition, this is **136 fits per outer fold** and **680 fits for one complete pass**, against 1,115 for the exhaustive equivalent — a 39% reduction with no loss of coverage, since every candidate is still evaluated at stage 1.

The label-permutation baseline repeats that pass 99 times, adding **67,320 fits**. The full analysis

is therefore approximately **68,000 model fits**, of which **99.0% belong to the null baseline** (Table 1).

Component	Fits	Share	Runtime
Nested search (real labels)	680	1.0%	—
Label-permutation null (99 repeats)	67,320	99.0%	—
Total, nested search	68,000		12.2 h (43,899 s)
Two-window importance analysis (RF, 10 seeds, 3 measures)	—		39 min
SHAP backfill	—		2.4 min
Block permutation (73 blocks, 10 seeds, 2 windows)	—		2.1 h
Verification pass from stored arrays	—		seconds

Table 1: Measured cost of the analysis. The label-permutation baseline accounts for 99% of the model fits and for the bulk of the runtime.

Against the measured wall-clock time this gives a mean of **646 ms per fit**, consistent with a shallow gradient-boosting model (maximum depth 1–2, 100–150 estimators) fitted to a 173×274 matrix on a single core.

The two-window analysis is a far smaller computation. It fits $2 \text{ windows} \times 5 \text{ folds} \times 10 \text{ seeds} = 100$ random forests of 300 trees each, or 30,000 trees; the run log records 39 minutes for the main pass, roughly 23 s per forest. Permutation importance over 274 features at 10 repeats per fitted model requires 274,000 scoring passes, but these are re-scorings rather than refits and are correspondingly cheap. The SHAP backfill, using TreeExplainer’s polynomial-time algorithm for tree ensembles, adds only 2.4 minutes. The block permutation extension of Section 7 refits the same 100 forests and evaluates 73 blocks rather than 274 individual features, at 2.1 hours. The verification pass of Section 5.4 rereads the stored arrays and completes in seconds, which is the point of storing them.

4.2 Where the cost is, and why it was accepted

The distribution above is the defining performance characteristic of this pipeline: **99% of the compute establishes a baseline rather than producing the result**. This was a deliberate trade rather than an oversight.

Without the null, the reported accuracy has no interpretation. The nested search selects among 37 windows and 2 configurations, and selection over that many candidates would be expected to produce some improvement over chance by chance alone. Only a baseline granted the same search freedom — which is why `null_use_full_grid` is set — establishes how much of the observed 0.737 is attributable to the data rather than to the search. Spending 99% of the budget to make the remaining 1% interpretable is the correct allocation.

4.3 Optimisation strategy: less work per repeat, not more workers

The pipeline is **entirely single-threaded**. The outer-fold loop and the 99-repeat null loop are plain Python for statements, and `n_jobs = 1` is set explicitly on every `permutation_importance` call rather than left at its default. Gradient boosting is inherently sequential in its estimators, so there is little to gain within a single fit; the parallelism available here is across fits, and none of it is exploited. (The separate random-forest analysis of Section 3.5 does use `n_jobs = -1`, since tree construction there is independent.)

Optimisation effort went instead into reducing the work per repeat, in two places. The two-stage candidate screen described in Section 4.1 removes 39% of the fits. More significantly, the null baseline function skips permutation importance, the Fisher-decorrelated ranking and the bootstrap confidence interval on every repeat, because only the out-of-fold balanced accuracy is needed to build the null distribution; the expensive interpretability machinery runs once, on the real labels. Both are algorithmic reductions rather than hardware ones, and both preserve the result exactly.

The unexploited opportunity is nonetheless the dominant cost. The 99 null repeats are mutually independent: each shuffles labels under its own seed, executes the same search, and returns a single scalar. There is no shared mutable state and no communication. Distributing them as a scheduler job array across N workers would reduce total runtime by close to a factor of N — with 24 workers, the twelve-hour job becomes roughly half an hour — at the cost of a trivial aggregation step reading 99 small files. This is the first change I would make, and the one with the largest return per line of code.

4.4 Execution environment

The analysis was executed on a departmental Linux server in a single Python 3.12 virtual environment, with SHAP v0.50.0 and scikit-learn providing the modelling and interpretability components. A full dependency listing is generated with `pip freeze` and distributed alongside the code, so that the environment can be reconstructed exactly rather than approximately. A single run occupies roughly twelve hours, so all state needed to interpret the result is written to disk on completion (Section 5) rather than held in a live session.

5 Reproducibility engineering

Given a twelve-hour runtime, reproducibility could not be an afterthought. The guiding requirement (R5) was that every number and figure in the scientific report should be regenerable without repeating the computation.

5.1 Machine-readable run records

Each run emits three JSON artefacts.

`run_metadata.json` records the input path, array shape, class counts, the full argument set (all window sizes, positions, crop values, band fractions, thresholds and repeat counts), the model

hyperparameter grid, and a summary of the label-score distribution. `summary.json` records every reported accuracy, confidence interval, selected window, feature ranking and null baseline. `reproducibility.json` records the per-seed accuracies, the split-half reproducibility statistics and the configuration that produced them.

Together these mean that the analysis outputs are self-describing. A reader encountering `summary.json` in isolation can reconstruct what was run, on what, with which settings — without access to the notebook, and without trusting a prose description that may have drifted from the code.

5.2 Seed management

Two seeding regimes are used, for different reasons.

The nested search is driven from a single root seed (42), from which the split, model-initialisation and permutation seeds are derived by fixed offsets, so that distinct stages are decorrelated while the whole run remains exactly reproducible. It is a point estimate, and reproducibility here means returning the identical number.

The two-window importance analysis instead sweeps ten seeds (0–9). Averaging stabilises the reported rankings, and the variation between seeds is what the split-half floor of Section 5.4 measures, so one sweep serves both. The split-half reproducibility floor is computed by partitioning those ten seeds into halves of five, averaging each half, and comparing the two resulting rankings; with all 126 distinct half-partitions evaluated and the median reported. Ten seeds yield 126 distinct half-partitions, which is enough for a stable median at a cost of ten repeats of a 39-minute analysis.

5.3 Recording known threats to validity in the artefact

`run_metadata.json` contains a field named `candidate_grid_provenance_caveat`. Its value is a plain-language note stating that the candidate grid — window sizes, progress-coordinate indices, crop values and band fractions — was narrowed using earlier non-nested exploratory runs, and that while the nested search is unbiased conditional on that grid, the grid itself was informed by hindsight.

This is unusual, and it is the practice in this codebase that would transfer most readily to other projects. A limitation recorded only in a report can become detached from the data it qualifies; recorded inside the output artefact, it travels with the results and cannot be lost. Anyone who later reuses `summary.json` — including me, months later — encounters the caveat at the point of use rather than having to remember it. The cost was one string field.

5.4 Independent recomputation from the stored artefacts

Because the analysis writes per-seed importance matrices rather than only the aggregated rankings, every reported statistic can be recomputed from the stored arrays without rerunning any model. This was done: a separate verification notebook reloads `fi_per_seed.npz` and regenerates the reproducibility floors, the cross-window correlations, the method-agreement matrix,

the composition counts and the leading-feature lists from scratch. The exercise found one error in an intermediate written summary — a top-15 Jaccard of 0.200 had been glossed as three shared features when it corresponds to five — and confirmed every other figure. It also revealed that the exported per-feature tables carried an all-null Shapley column, because that measure was computed in a later backfill that never wrote back to them; the tables were regenerated from the arrays.

Storing intermediate arrays rather than only summaries converts verification from a rerun into a read. The six per-seed matrices occupy about 0.13 MB between them, so the store costs essentially nothing, and it is what made this audit possible at all.

5.5 Separation of computation from presentation

Figure generation is separated from analysis. The expensive pipeline writes numerical artefacts; a second, fast notebook reads those artefacts and produces figures. Consequently a change to a figure’s styling, axis range or colour scale costs seconds rather than twelve hours. The separation does not by itself keep artefacts in agreement — the all-null Shapley column of Section 5.4 arose because a later computation updated one and not the other — but it does mean any such discrepancy is repairable from the stored arrays rather than by rerunning.

6 Testing and validation

The pipeline has no unit test suite. This section describes the validation it does have — all of it at the level of whole-run outputs rather than of individual functions — and is explicit about what is missing.

6.1 Input validation

Fourteen guard clauses raise on malformed input before any model is fitted. At ingestion these check that the CV array is three-dimensional, that the label and group vectors match its first dimension, that it contains no NaN or infinite values, and that every label is one of the two expected strings; further clauses reject window sizes larger than the trajectory, windows left empty by the frame stride, and crops that leave no usable trajectory. These catch the failure mode most likely to arise in practice — a mis-joined HDF5 file or a mistyped window specification — at the point of entry rather than several hours into a run. Separately, the run record stores whether any trajectory group carried inconsistent labels and whether end-state scores were present; both are recorded rather than enforced, so a reader of the outputs can check them after the fact.

6.2 The permutation null as an integration test

As described in Section 3.3, the null baseline centred on 0.497 ± 0.040 is a direct empirical check that the splitting and nesting logic behave as designed. This is the closest thing in the codebase to a correctness test of the core algorithm, and it tests the pipeline end to end rather than any component in isolation.

6.3 Cross-method agreement as validation

Computing three feature-importance measures independently provides a further check. An implementation error in any one of them — an index misalignment between feature columns and names, for instance, or importances computed on training rather than held-out rows — would show up as an inexplicable disagreement with the other two. The observed pattern is instead structured and explicable: impurity and Shapley importances agree at $\rho = 0.972$ and 0.975 in the two windows, because for tree ensembles both derive from the same splits, while permutation importance diverges for a documented statistical reason rather than a computational one. A result that is anomalous but explicable is very different from one that is merely anomalous, and distinguishing the two is a validation activity.

The verification pass of Section 5.4 is the strongest form this check took. Every importance statistic was recomputed from the stored per-seed arrays by code written separately from the pipeline that produced them, and the two agreed on all but one figure — which proved to be an error in a written summary rather than in either implementation.

6.4 What is missing

Beyond the absence of a unit test suite already noted, there is no continuous integration and no static type checking, and the code lives in notebooks rather than in an importable package. The verification notebook of Section 5.4 is a regression check on the outputs, not a test of the components that produce them.

Were I to add tests, I would start with the two functions where a silent error would be most damaging and least visible: the windowed aggregation, and the group-splitting logic. For the first, a test would construct a small synthetic array with known per-window means and assert that the aggregation recovers them, including at window boundaries and under the frame stride. For the second, a test would assert that no group identifier appears in both sides of any split, across many random seeds — a property that is cheap to check exhaustively and catastrophic to get wrong. Both failure modes produce plausible-looking output, which is precisely why they warrant tests rather than inspection.

7 Limitations and future engineering work

Packaging. The analysis is implemented across Jupyter notebooks with shared configuration passed by argument dictionaries. This suited exploratory development but impedes reuse and testing. Refactoring the aggregation, splitting and importance routines into an importable module, with the notebooks reduced to thin driver scripts, is the first step towards both.

Memory model. The dataset is loaded whole. At approximately 1 GB in double precision this is acceptable on the machine used, but the approach does not scale: an ensemble ten times larger would not fit. Because the pipeline only ever needs windowed means, a chunked reader computing aggregates incrementally would remove the constraint without changing any downstream code.

Parallelisation of the null baseline. As set out in Section 4.3, this is the single largest available speed-up and requires no algorithmic change.

Block permutation importance. The scientific analysis found permutation importance unusable in one regime because correlated coordinates mask one another. The algorithmic response is to permute groups of correlated features jointly rather than individually. This was implemented as a separate module and executed: hierarchical clustering on $1 - |\rho|$ reduced the 274 coordinates to 73 blocks, and recomputing block-level importance over the same ten seeds took 2.1 hours. Two implementation details determine whether the result is meaningful. Blocks are shared between the two analysis windows and built from the *minimum* distance across them, because averaging the two correlation matrices would fail to merge exactly the groups that are correlated in only one window. Within a block, all columns are permuted under a common row ordering, so that the internal correlation structure survives and only the association with the label is severed.

The outcome is a partial success and is reported as such in the dissertation: the late-window reproducibility floor rose from a top-15 Jaccard of 0.034 to 0.200 and the fraction of exactly-zero importances fell from 42% to 36%, but the resulting block importances are confounded with block size, which ranges from 1 to 74 coordinates. Normalising importance by block size, or constraining the clustering to produce comparable block sizes, is the obvious next step and requires no new machinery.

Configuration handling. Configuration is currently passed as function arguments and echoed into the metadata JSON. Moving to an external configuration file read at startup would remove the possibility of a discrepancy between what was intended and what was recorded, and would make parameter sweeps easier to script.

8 Repository guide

The code is available at <https://github.com/171172701a/mltsa-trypsin-msc2025-2026>. Table 2 maps its contents to the sections of this report.

Path	Purpose
<code>run_improved_pipeline_v4.ipynb</code>	Fixed window and hyperparameter search; sections A–D of the analysis
<code>window_fi_contrast.ipynb</code>	Two fixed windows, three importance measures, split-half floor
<code>window_fi_figures.ipynb</code>	Figure generation from serialised artefacts
<code>block_permutation.ipynb</code>	Block permutation importance; 73 blocks, 2.1 h runtime
<code>verify_fi.ipynb</code>	Recomputes every reported importance statistic from <code>fi_per_seed.npz</code>
<code>fi_per_seed.npz</code>	Per-seed importance matrices, three measures <i>times</i> two windows
<code>block_perm_summary.json</code>	Block-level floors, cross-window agreement, water enrichment
<code>feature_names.csv</code>	Collective-variable index to residue or water-rank mapping
<code>run_metadata.json</code>	Full configuration, dataset summary, provenance caveat
<code>summary.json</code>	All reported accuracies, intervals and feature rankings
<code>reproducibility.json</code>	Per-seed accuracies and split-half floors
<code>requirements.txt</code>	Pinned dependencies (single environment, Python 3.12)

Table 2: Mapping of repository contents to the sections of this report.

The trajectory dataset is not redistributed; it is available from the project supervisor.

9 Closing remark

Three of the software decisions described above turned out to carry scientific weight rather than only engineering weight. Aggregating each trajectory to a single row was adopted for correctness, and it is the first of the three layers on which the credibility of the reported accuracies rests. The label-permutation baseline was built as a statistical control, and its centring on 0.497 is the direct evidence that the grouping logic is sound. Storing per-seed importance matrices rather than only their averages cost 0.13 MB and was what allowed every reported statistic to be recomputed afterwards, an exercise that located an error in an intermediate summary and would otherwise have cost a twelve-hour rerun.

In this kind of analysis the engineering choices and the scientific ones are not separable: a pipeline that is fast but leaks does not produce a weaker result, it produces one that cannot be evaluated. All figures reported in the accompanying dissertation can be regenerated from the serialised JSON and NPZ artefacts alone, without repeating the twelve-hour search.